



ITES
RENÉ DESCARTES

Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutiérrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Castro Blanco Manuel David

Fecha:
20 de Julio 2026

Tabla de contenido

<i>Introducción</i>	<i>3</i>
<i>Identificación de problemas</i>	<i>3</i>
<i>Análisis de Clean Code.....</i>	<i>8</i>
<i>Aplicación de los principios SOLID</i>	<i>10</i>
<i>Análisis de la deuda técnica.....</i>	<i>12</i>
<i>Definición del MVP (Producto Mínimo Viable)</i>	<i>13</i>
<i>Definition of Done (DoD)</i>	<i>14</i>
<i>Conclusión</i>	<i>15</i>

Introducción

El siguiente documento tiene como propósito realizar una auditoría de calidad sobre el sistema "GymManager", una plataforma web desarrollada por la empresa GymManager Solutions para la administración integral de un gimnasio. El sistema abarca módulos de registro de clientes, control de membresías, registro de pagos, gestión de entrenadores, reservación de clases, reportes administrativos y control de asistencia.

Tras ocho meses de desarrollo, el proyecto comenzó a presentar retrasos y un aumento considerable en los costos de mantenimiento. Actuando como consultor de Ingeniería de Software, el objetivo de este informe es analizar el estado actual del sistema desde la perspectiva de la calidad del desarrollo, identificar las malas prácticas de programación, diseño y gestión que lo afectan, y proponer mejoras concretas sustentadas en los principios de Clean Code y SOLID.

Para ello, el documento identifica los problemas presentes en el proyecto, los analiza a la luz de los principios de código limpio y de diseño orientado a objetos, examina la deuda técnica acumulada, define un Producto Mínimo Viable (MVP) ante las nuevas funcionalidades solicitadas por el cliente y establece una Definition of Done que sirva como criterio formal de terminación. Finalmente, se presenta una reflexión sobre la importancia de estos temas para el desarrollo de software de calidad.

Identificación de problemas

A partir de la auditoría realizada al sistema GymManager se identificaron los siguientes problemas. Para cada uno se describe qué ocurre, por qué representa una mala práctica y qué consecuencias puede generar durante el desarrollo o el mantenimiento del sistema.

Problema 1. Clases con más de 1,800 líneas de código

Qué ocurre: Existen clases que superan las 1,800 líneas de código.

Por qué es mala práctica: Una clase tan extensa concentra demasiadas responsabilidades y viola el principio de responsabilidad única. Se vuelve difícil de leer, entender y probar, y cualquier cambio obliga a navegar cientos de líneas para localizar la lógica relevante.

Consecuencias: Aumenta el tiempo necesario para comprender el código, se eleva la probabilidad de introducir errores al modificarlo y se dificulta la realización de pruebas unitarias, lo que incrementa el costo de mantenimiento.

Problema 2. Funciones con más de 250 líneas

Qué ocurre: Algunas funciones contienen más de 250 líneas de código.

Por qué es mala práctica: Las funciones deben ser pequeñas y hacer una sola cosa. Una función tan larga mezcla múltiples niveles de abstracción y flujos de control, lo que la hace prácticamente imposible de seguir.

Consecuencias: La depuración se vuelve lenta y propensa a errores, resulta complicado reutilizar la lógica y las pruebas no pueden aislar comportamientos concretos.

Problema 3. Código duplicado mediante copiar y pegar

Qué ocurre: Se reutiliza código copiando y pegando entre distintos módulos.

Por qué es mala práctica: Incumple el principio DRY (Don't Repeat Yourself). La misma lógica existe en varios lugares sin una fuente única de verdad.

Consecuencias: Cuando se corrige un error o se cambia una regla, hay que modificar todas las copias; olvidar una genera comportamientos inconsistentes y errores difíciles de rastrear.

Problema 4. Nombres de variables poco descriptivos

Qué ocurre: Los nombres de variables son poco descriptivos, como x, dato, temp, valor1 u obj.

Por qué es mala práctica: El código debe comunicar su intención. Nombres genéricos obligan al lector a deducir el propósito de cada variable a partir del contexto.

Consecuencias: Se reduce la legibilidad, aumenta el tiempo de comprensión para cualquier desarrollador nuevo y se incrementa el riesgo de usar la variable incorrecta.

Problema 5. Un mismo archivo con múltiples responsabilidades

Qué ocurre: Un mismo archivo realiza validaciones, consultas a la base de datos, genera reportes y envía correos electrónicos.

Por qué es mala práctica: Concentra responsabilidades que deberían estar separadas en capas o componentes distintos, acoplando lógica de negocio, acceso a datos y presentación.

Consecuencias: Un cambio en una responsabilidad puede romper las demás, se dificulta la reutilización y las pruebas se vuelven complejas porque no es posible aislar cada función.

Problema 6. Estructura de proyecto inconsistente entre desarrolladores

Qué ocurre: Cada desarrollador utiliza una estructura diferente para organizar el proyecto.

Por qué es mala práctica: La falta de una organización común impide predecir dónde se encuentra cada elemento del sistema.

Consecuencias: Se pierde tiempo buscando archivos, se dificulta la incorporación de nuevos integrantes al equipo y aumenta el desorden general del código.

Problema 7. Ausencia de una guía de estilos de programación

Qué ocurre: No existe una guía de estilos de programación.

Por qué es mala práctica: Sin convenciones compartidas, cada quien escribe de forma distinta, generando un código heterogéneo y difícil de mantener.

Consecuencias: El código carece de uniformidad, las revisiones se complican y las decisiones sobre formato o nomenclatura se discuten una y otra vez sin criterio común.

Problema 8. Comentarios que explican cada línea en lugar de código claro

Qué ocurre: Los comentarios explican lo que hace cada línea de código en lugar de escribir un código claro por sí mismo.

Por qué es mala práctica: El buen código debe ser autoexplicativo. Un exceso de comentarios suele ser síntoma de que el código no es legible.

Consecuencias: Los comentarios se desactualizan respecto al código y terminan generando confusión; además, no resuelven el problema de fondo de la baja legibilidad.

Problema 9. Funcionalidades consideradas terminadas cuando "parecen funcionar"

Qué ocurre: Las funcionalidades se consideran terminadas únicamente cuando "parecen funcionar".

Por qué es mala práctica: No existe un criterio objetivo ni verificable de terminación; "parecer funcionar" no garantiza que el código sea correcto, esté probado o cumpla los requisitos.

Consecuencias: Se liberan funciones con errores ocultos, se acumulan defectos y disminuye la confianza en el sistema.

Problema 10. Ausencia de revisiones de código entre compañeros

Qué ocurre: Nunca se realizan revisiones de código entre compañeros.

Por qué es mala práctica: La revisión por pares es una de las prácticas más efectivas para detectar errores tempranamente y difundir conocimiento en el equipo.

Consecuencias: Los errores llegan a producción, el conocimiento queda aislado en cada desarrollador y no se comparten buenas prácticas.

Problema 11. Alto acoplamiento: cada funcionalidad obliga a modificar varias clases

Qué ocurre: Cada nueva funcionalidad requiere modificar varias clases existentes.

Por qué es mala práctica: Evidencia un diseño rígido y fuertemente acoplado que incumple el principio abierto/cerrado.

Consecuencias: Cada cambio es costoso y arriesgado, se rompen funcionalidades ya existentes y el sistema se vuelve frágil ante cualquier evolución.

Problema 12. Desarrollo de funcionalidades no solicitadas por el cliente

Qué ocurre: Se agregaron funcionalidades que el cliente nunca solicitó porque el equipo pensó que "serían útiles".

Por qué es mala práctica: Se invierte esfuerzo en características sin valor comprobado para el cliente, desviando recursos de lo prioritario (sobreingeniería).

Consecuencias: Se desperdicia tiempo y presupuesto, se agrega complejidad innecesaria y se retrasa la entrega de lo que realmente importa.

Problema 13. Errores conocidos que no se corrigen

Qué ocurre: Existen errores conocidos que no se corrigen para poder avanzar más rápido.

Por qué es mala práctica: Posponer indefinidamente la corrección de defectos genera deuda técnica y deteriora la calidad del producto.

Consecuencias: Los errores se acumulan, se vuelven más costosos de resolver con el tiempo y afectan la experiencia del usuario final.

Problema 14. Documentación desactualizada

Qué ocurre: La documentación del proyecto nunca se actualiza.

Por qué es mala práctica: Una documentación obsoleta es tan perjudicial como no tenerla, porque induce a error a quien la consulta.

Consecuencias: Los nuevos integrantes se confunden, se toman decisiones sobre información incorrecta y se pierde trazabilidad del sistema.

Problema 15. Ausencia de criterio formal para aprobar funcionalidades

Qué ocurre: No existe ningún criterio formal para aprobar una funcionalidad antes de integrarla al proyecto principal.

Por qué es mala práctica: Sin una Definition of Done ni controles de integración, cualquier código puede incorporarse sin cumplir estándares mínimos.

Consecuencias: Se integra código de baja calidad, se desestabiliza la rama principal y aumentan los errores en el producto final.

Análisis de Clean Code

De los problemas identificados se seleccionan cinco directamente relacionados con la calidad del código. Para cada uno se indica el principio de Clean Code que se incumple, cómo debería corregirse y qué beneficios aportaría aplicar la mejora.

1. Nombres poco descriptivos (x, dato, temp, valor1, obj)

Principio incumplido: "Usar nombres significativos e intencionados". Clean Code establece que los nombres deben revelar la intención y responder qué representa la variable, por qué existe y cómo se usa.

Cómo corregirse: Renombrar las variables con nombres que describan su propósito, por ejemplo clienteActivo, totalPago, fechaVencimiento o listaMembresias, evitando abreviaturas ambiguas.

Beneficios: El código se vuelve autoexplicativo, se reduce el tiempo de comprensión, disminuyen los errores por confusión de variables y se facilita el mantenimiento.

2. Funciones con más de 250 líneas

Principio incumplido: "Las funciones deben ser pequeñas y hacer una sola cosa". Una función debe tener un único nivel de abstracción y una responsabilidad clara.

Cómo corregirse: Dividir la función extensa en varias funciones pequeñas y bien nombradas, extrayendo cada bloque lógico (validación, cálculo, persistencia) a su propia función reutilizable.

Beneficios: Mejora la legibilidad, permite reutilizar lógica, facilita las pruebas unitarias y simplifica la depuración al aislar comportamientos.

3. Código duplicado (copiar y pegar entre módulos)

Principio incumplido: DRY (Don't Repeat Yourself). El conocimiento debe tener una representación única y sin ambigüedad dentro del sistema.

Cómo corregirse: Extraer la lógica repetida a funciones, clases o servicios comunes y reutilizarlos desde los distintos módulos en lugar de duplicar el código.

Beneficios: Un cambio se realiza en un solo lugar, se elimina la inconsistencia entre copias, se reduce el tamaño del código y baja el costo de mantenimiento.

4. Comentarios que explican cada línea en lugar de código claro

Principio incumplido: "El código debe explicarse por sí mismo; los comentarios no deben suplir un mal código". Clean Code prefiere código expresivo antes que comentarios redundantes.

Cómo corregirse: Reescribir el código para que sea legible mediante nombres claros y funciones pequeñas, eliminando los comentarios obvios y conservando solo los que aporten contexto o justifiquen decisiones no evidentes.

Beneficios: Se evita que los comentarios queden desactualizados, el código comunica su intención por sí mismo y se reduce el ruido visual.

5. Un mismo archivo realiza validaciones, consultas, reportes y correos

Principio incumplido: "Separación de responsabilidades" (base del principio de responsabilidad única aplicado a Clean Code). Cada módulo debe tener una única razón para cambiar.

Cómo corregirse: Separar las responsabilidades en capas o clases distintas: una para validación, otra para acceso a datos, otra para generación de reportes y otra para el envío de correos.

Beneficios: El sistema se vuelve modular, cada componente puede probarse y mantenerse de forma independiente, y un cambio en una responsabilidad no afecta a las demás.

Aplicación de los principios SOLID

A continuación se analizan los principios SOLID que no están siendo respetados en el sistema GymManager. No es obligatorio encontrar los cinco; se identifican únicamente aquellos que realmente aplican al caso, justificando cada uno con la evidencia correspondiente.

S — Principio de Responsabilidad Única (SRP)

En qué consiste: Una clase debe tener una sola responsabilidad, es decir, una única razón para cambiar.

Evidencia en el caso: Un mismo archivo realiza validaciones, consultas a la base de datos, genera reportes y envía correos; además existen clases de más de 1,800 líneas que concentran múltiples responsabilidades.

Cambios a realizar: Dividir esas clases y archivos en componentes especializados: una capa de validación, una de acceso a datos (repositorios), un servicio de reportes y un servicio de notificaciones por correo.

Beneficios: Cada clase se vuelve más pequeña, comprensible y fácil de probar; los cambios quedan localizados y se reduce el riesgo de romper funcionalidades no relacionadas.

O — Principio Abierto/Cerrado (OCP)

En qué consiste: El software debe estar abierto a la extensión pero cerrado a la modificación; se debe poder agregar comportamiento sin alterar el código existente.

Evidencia en el caso: Cada nueva funcionalidad requiere modificar varias clases existentes, lo que demuestra que el sistema no permite extenderse sin cambiar lo ya construido.

Cambios a realizar: Introducir abstracciones (interfaces o clases base) y aplicar patrones como estrategia o inyección de dependencias, de modo que las nuevas funcionalidades se agreguen mediante nuevas clases en lugar de modificar las existentes.

Beneficios: Se reduce el riesgo de introducir errores en código estable, se facilita la evolución del sistema y se agiliza la incorporación de nuevas características.

D — Principio de Inversión de Dependencias (DIP)

En qué consiste: Los módulos de alto nivel no deben depender de los de bajo nivel; ambos deben depender de abstracciones. Los detalles deben depender de las abstracciones y no al revés.

Evidencia en el caso: Al mezclar en un mismo archivo la lógica de negocio con el acceso directo a la base de datos y el envío de correos, la lógica de alto nivel queda acoplada a detalles concretos de implementación.

Cambios a realizar: Definir interfaces para el acceso a datos y para el envío de correos, e inyectar esas dependencias en la lógica de negocio, de modo que dependa de abstracciones y no de implementaciones concretas.

Beneficios: Se reduce el acoplamiento, se pueden sustituir implementaciones (por ejemplo, cambiar el proveedor de correo) sin tocar la lógica de negocio y se facilita el uso de dobles de prueba (mocks) en las pruebas.

Los principios de sustitución de Liskov (L) e segregación de interfaces (I) no cuentan con evidencia suficiente en la información proporcionada del caso, por lo que no se incluyen en el análisis; el enfoque se centra en los tres principios anteriores, que sí se manifiestan claramente en los problemas detectados.

Análisis de la deuda técnica

La deuda técnica representa el costo futuro derivado de decisiones de desarrollo que priorizan la rapidez sobre la calidad. En el sistema GymManager, varias decisiones generan deuda técnica, entre ellas: mantener errores conocidos sin corregir, duplicar código en lugar de reutilizarlo, no actualizar la documentación, omitir las revisiones de código y considerar terminadas funciones que solo "parecen funcionar".

¿Qué tipo de deuda técnica se observa? Se observa principalmente deuda técnica deliberada y no gestionada. Es deliberada porque el equipo decide conscientemente avanzar más rápido dejando errores sin corregir y omitiendo buenas prácticas; y es no gestionada porque no se registra ni se planifica su pago, acumulándose de forma descontrolada. También hay deuda de código (duplicación, clases y funciones enormes), de documentación (documentación desactualizada) y de pruebas (ausencia de criterios de terminación y de revisiones).

¿Cómo afectará al mantenimiento del sistema? El mantenimiento se vuelve cada vez más costoso y lento: cada cambio exige modificar múltiples clases acopladas, corregir defectos que se propagan por el código duplicado y trabajar sobre una documentación que no refleja el estado real del sistema. Con el tiempo, el equipo dedica más esfuerzo a mantener lo existente que a desarrollar nuevas funcionalidades.

¿Qué impacto tendrá sobre el tiempo y el costo del desarrollo? El impacto es directo: los plazos se alargan porque cada tarea toma más tiempo del previsto, y los costos aumentan debido al retrabajo, la corrección de errores acumulados y la dificultad para incorporar cambios. Esto coincide con la situación descrita en el caso, donde tras ocho meses el proyecto se retrasó y sus costos de mantenimiento crecieron considerablemente.

¿Qué acciones se proponen para disminuir la deuda? Se proponen las siguientes acciones: refactorizar de forma progresiva las clases y funciones extensas; eliminar el código duplicado extrayéndolo a componentes reutilizables; establecer y aplicar una guía de estilos y una Definition of Done; corregir los errores conocidos priorizándolos por

impacto; implementar revisiones de código entre compañeros; mantener la documentación actualizada; e introducir pruebas automatizadas. Además, conviene registrar la deuda técnica en el backlog para gestionarla y pagarla de manera planificada.

Definición del MVP (Producto Mínimo Viable)

Ante las nuevas funcionalidades solicitadas por el cliente, como líder del proyecto se determina cuáles forman parte del Producto Mínimo Viable y cuáles se posponen para futuras versiones. El MVP debe incluir únicamente lo indispensable para que un gimnasio opere y genere valor desde el primer momento, priorizando el valor para el cliente, el tiempo de desarrollo y el impacto en el negocio.

Funcionalidades que forman parte del MVP:

- Gestión de clientes: es la base del sistema; sin el registro de clientes no puede operar ningún otro módulo.
- Registro de pagos: representa el flujo de ingresos del gimnasio, esencial para el negocio.
- Control de asistencia: permite dar seguimiento operativo diario a los clientes que acuden al gimnasio.
- Reservación de clases: funcionalidad central para organizar el servicio que ofrece el gimnasio.
- Reportes básicos de ingresos: aportan información mínima para la toma de decisiones administrativas.

Funcionalidades que se dejan para futuras versiones:

- Chat interno entre entrenadores: útil, pero no esencial para operar; puede resolverse temporalmente con otras herramientas.
- Integración con relojes inteligentes: agrega complejidad técnica alta y no es indispensable para el funcionamiento básico.

- Sistema de recompensas: es una mejora de fidelización que puede esperar a versiones posteriores.
- Publicación automática en redes sociales: función de marketing secundaria frente a la operación del gimnasio.
- Tienda en línea de suplementos: constituye un módulo de comercio electrónico independiente, con su propia complejidad, que excede el alcance mínimo.

Justificación general: el MVP se concentra en las funciones que permiten administrar clientes, cobrar y ofrecer el servicio del gimnasio, que son las que generan valor inmediato y sostienen el negocio con el menor tiempo de desarrollo. Las funcionalidades diferidas, aunque atractivas, aportan valor complementario, incrementan la complejidad y el tiempo de entrega, y pueden incorporarse de forma incremental una vez validado y estabilizado el núcleo del sistema.

Definition of Done (DoD)

La siguiente Definition of Done establece los criterios que una funcionalidad debe cumplir antes de considerarse terminada. Cada criterio está redactado de forma clara y verificable, y abarca calidad del código, funcionamiento, pruebas, documentación, integración, revisión de código y cumplimiento de requerimientos. Una funcionalidad se considera terminada solo cuando cumple todos los siguientes criterios:

- 1. El código cumple con la guía de estilos definida por el equipo y utiliza nombres descriptivos.
- 2. La funcionalidad cumple con todos los requerimientos acordados con el cliente y su criterio de aceptación.
- 3. La funcionalidad fue probada y funciona correctamente en el entorno de pruebas, sin errores conocidos pendientes.
- 4. Se implementaron pruebas (unitarias o de aceptación) y todas se ejecutan de forma satisfactoria.

- 5. El código no contiene duplicaciones evitables y respeta el principio de responsabilidad única.
- 6. El código fue revisado y aprobado por al menos un compañero mediante revisión de código.
- 7. La funcionalidad se integró en la rama principal sin romper las funcionalidades existentes.
- 8. La documentación relacionada (técnica y de usuario) fue actualizada conforme a los cambios realizados.
- 9. Se verificó que la funcionalidad no introduce regresiones en otros módulos del sistema.
- 10. La funcionalidad fue validada y aprobada formalmente antes de integrarse al proyecto principal.

Conclusión

¿Cuál de los temas vistos consideras más importante para desarrollar software de calidad? Entre todos los temas analizados, considero que los principios de Clean Code son los más importantes para desarrollar software de calidad, ya que constituyen la base sobre la que se sostienen las demás prácticas. Un código limpio, legible y bien estructurado facilita aplicar los principios SOLID, reduce la deuda técnica y hace posible definir criterios claros de terminación. Sin código limpio, cualquier otra buena práctica resulta difícil de mantener en el tiempo.

¿Qué consecuencias tendría ignorar estos principios en un proyecto real? Ignorar estos principios conduce exactamente a la situación del sistema GymManager: retrasos, aumento de costos de mantenimiento, acumulación de errores, código frágil y difícil de modificar, y un equipo que invierte más esfuerzo en corregir que en avanzar. A largo plazo, esto puede provocar la pérdida de la confianza del cliente e incluso el fracaso del proyecto.

¿Cómo aplicarás estos conocimientos en tu propio proyecto de la asignatura? Aplicaré estos conocimientos escribiendo desde el inicio un código con nombres claros y funciones pequeñas, evitando la duplicación y separando responsabilidades. Definiré una Definition of Done para no considerar terminada una función hasta que esté probada y documentada, delimitaré un MVP para concentrar el esfuerzo en lo que aporta valor y gestionaré conscientemente la deuda técnica para evitar que se acumule. De esta forma, mi proyecto será más mantenible, confiable y de mayor calidad.